Part I

Power Programming
Bitwise Tips and Tricks

If you are a seasoned programmer, these tips and tricks will seem very familiar, and are probably already part of your repertoire. If you're a novice programmer or a student, they should help you experience an "Aha!" moment. Independent of what you currently do, these tips and tricks will remind you of the wonderful discoveries in computer science, and the brilliant men and women behind them.

Before we get started, let's establish some conventions for the rest of the article. Figure 1 shows how we represent bits—we start from right to left. The rightmost bit is the 'Least Significant Bit' (LSB), and labelled as b_0. The 'Most Significant Bit' (MSB) is labelled b_n. We use 8 bits to indicate and demonstrate concepts. The concept, however, is generically applicable to 32, 64, and even more bits.

Population count

Population count refers to the number of bits that are set to 1. Typical uses of population counting are:

- Single-bit parity generation and detection: To generate the correct parity bit, depending on the scheme being followed (odd or even parity), one would need to count the number of bits set to 1, and generate the corresponding bit for parity. Similarly, to check the parity of a block of bits, we would need to check the number of 1s, and validate the block against the expected parity setting.
- Hamming weight: Hamming weight is used in several fields, ranging from cryptography to information theory. The hamming distance between two strings A and B can be computed as the hamming weight of "A" XOR "B".

These are just a few of the use cases of population counting; we cannot hope to cover all possible use cases, but rather, just explore a few samples.

First implementation

Our first implementation is the most straightforward:

```
int count_ones(int num)
{
  int count = 0;
  int mask = 0x1;
  while (num) {
    if (num & mask)
      count++;
    num >>= 1;
  }
  return count;
}
```

The code creates a mask bit, which is the number 1. The number is then shifted right, one at a time, and checked to see if the bit just shifted to the rightmost bit (LSB), is set. If so, the count is incremented. This technique is rather rudimentary, and has a cost complexity of $O(n)$, where n is the number of bits in the block under consideration.

Improving the algorithm

For those familiar with design techniques like divide and conquer, the idea below is a classical trick called the 'Gillies-Miller method for sideways